

B.TECH PROJECT REPORT | JULY 2026

# Handwritten Digit Recognition

*Using Deep Neural Networks*

---

**Vaidehi Aggarwal**

B.Tech (Computer Science Engineering) — 5th Semester

Vivekananda Institute of Professional Studies, GGSIPU

# What We'll Cover

01

## The Problem

Why automated digit recognition matters across industries

02

## The Approach

MNIST dataset, neural network architecture & training

03

## The Results

96.86% test accuracy, per-class analysis, live predictions

04

## What's Next

Limitations, applications & future enhancements

## THE PROBLEM

# Manual Digit Processing Doesn't Scale

Handwritten digits still drive core workflows across major industries — and reading them by hand is slow, costly, and error-prone.

**2-5%**

Bank cheque error rate from manual digit reading

**94-96%**

Accuracy ceiling of traditional hand-crafted feature methods

**Millions**

Of postal items & exam papers processed manually every day

### AFFECTED SECTORS

1

Banking

2

Postal Services

3

Education

4

Healthcare

# What This Project Set Out To Do

01

## Design

Build a neural network achieving >95% accuracy on the MNIST test set (10,000 unseen images).

02

## Deploy

Develop a full pipeline: preprocessing, training, evaluation, and inference on custom images.

03

## Validate

Test the system on real, user-drawn digits beyond the standard MNIST benchmark.

04

## Analyze

Conduct rigorous performance analysis — confusion matrices, per-class metrics, error patterns.

## THE DATASET

# MNIST: The Gold-Standard Benchmark

**70,000**

Total grayscale images

**60,000**

Training samples

**10,000**

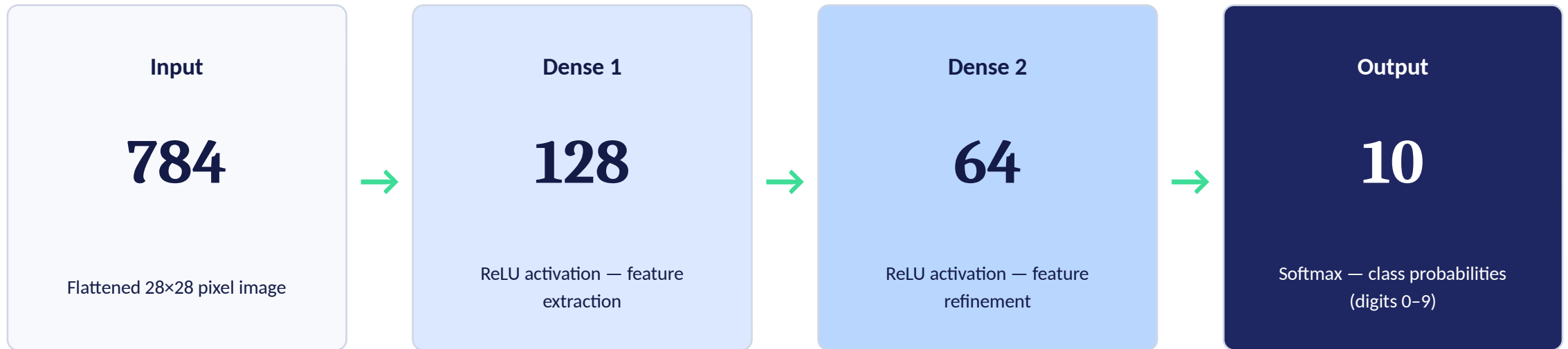
Test samples

**28×28**

Pixel resolution

- 
- Compiled by LeCun, Cortes & Burges (1998) from NIST Special Databases 1 & 3
  - 250 contributors — Census Bureau employees and high-school students — for natural handwriting variation
  - Each digit class holds ~6,000–7,000 balanced training samples
  - Pixel values 0–255, normalized to [0, 1] before training

# Sequential Neural Network Architecture



**Optimizer:** Adam (lr = 0.001) **Loss:** Sparse Categorical Crossentropy **Params:** ~109,000

IMPLEMENTATION

# Training Configuration

|                  |                                           |
|------------------|-------------------------------------------|
| Environment      | Google Colab (free GPU: NVIDIA T4 / P100) |
| Epochs           | 10 full passes over the training set      |
| Batch size       | 128 samples per weight update             |
| Validation split | 20% of training data held out             |
| Framework        | TensorFlow 2.x / Keras Sequential API     |

```
Importing the Dependencies

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import cv2
from google.colab.patches import cv2_imshow
from PIL import Image
import tensorflow as tf
tf.random.set_seed(3)
from tensorflow import keras
from keras.datasets import mnist
from tensorflow.math import confusion_matrix

Loading the MNIST data from keras.datasets

(X_train, Y_train), (X_test, Y_test) = mnist.load_data()

type(X_train)

numpy.ndarray

# shape of the numpy arrays
print(X_train.shape, Y_train.shape, X_test.shape, Y_test.shape)

(60000, 28, 28) (60000,) (10000, 28, 28) (10000,)

Training data = 60,000 Images
Test data = 10,000 Images
```

Figure — Loading dependencies & the MNIST dataset in Google Colab

## THE RESULTS

# Objective Exceeded

98.9%

Training Accuracy

96.86%

Test Accuracy

95%

Project Target

*A gap of only 2.04 percentage points between training and test accuracy shows the model generalizes well — it learned real digit features, not the training set by heart.*



# Accuracy Varies Slightly by Digit



## WHY 4 & 9 LAG SLIGHTLY

Digits 4 and 9 share a visually similar upper curve structure, so the model occasionally confuses one for the other.

Even the weakest classes stay above 94% — well within acceptable range for practical deployment.

# Beyond the Benchmark: Custom Digits

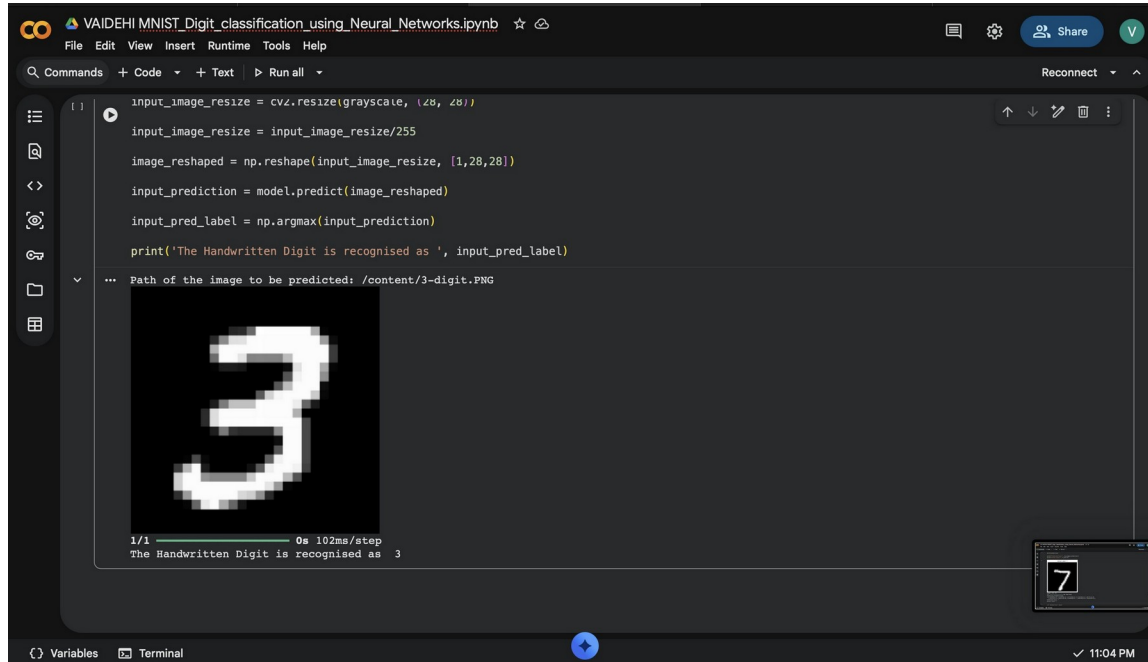


Figure — Custom handwritten '3' correctly recognized in Google Colab

**~102ms inference time per image on GPU — fast enough for real-time use.**

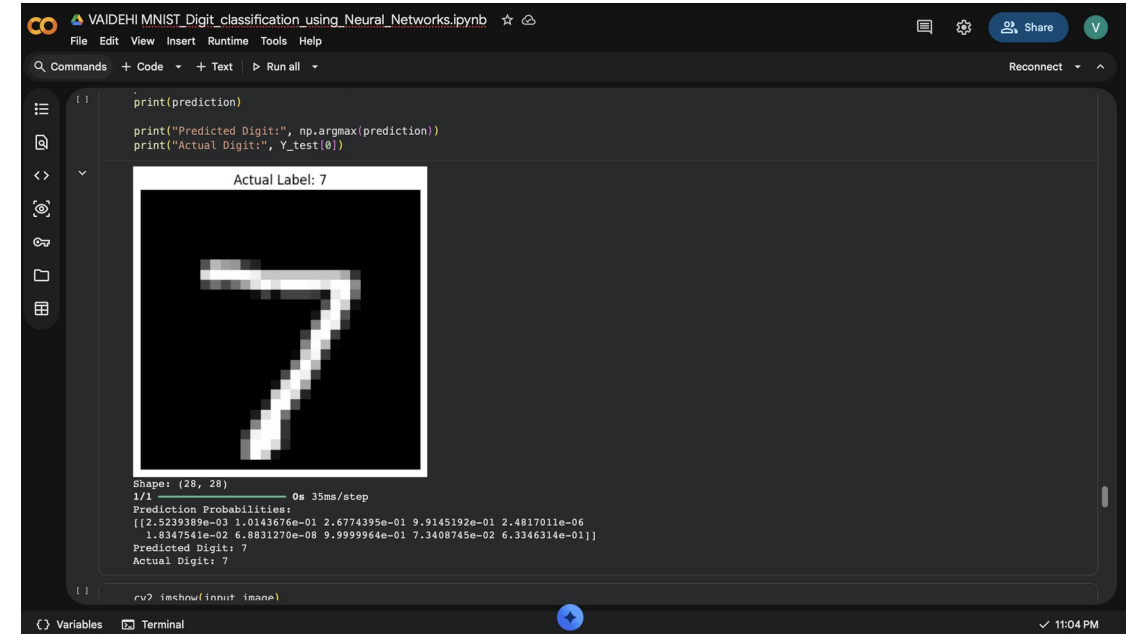


Figure — Prediction verified against the actual label on a held-out test sample

# Where This Technology Applies



## Banking

Cheque amount & account number recognition,  
MICR scanning



## Postal & Logistics

ZIP code reading for automated mail sorting



## Education

Automated grading of numeric exam responses



## Healthcare

Digitizing handwritten prescriptions & patient  
records



## Document Archives

Converting historical handwritten records to  
searchable text



## OCR Systems

Foundational component for full character  
recognition

# Limitations & Future Enhancements

## CURRENT LIMITATIONS

- Single isolated digits only — no multi-digit sequences
- Fixed 28×28 grayscale input format
- No confidence/uncertainty estimation
- Limited interpretability ("black box" predictions)

## PLANNED ENHANCEMENTS

- Convolutional layers to push accuracy toward 99%+
- Data augmentation (rotation, scaling, elastic distortion)
- Multi-digit sequence recognition via CTC
- Model compression for mobile / edge deployment

## CONCLUSION

# Deep Learning Delivers on Its Promise

A Sequential neural network — no manual feature engineering required — reached 96.86% test accuracy, exceeding the 95% target, and correctly recognized real, user-drawn digits beyond the MNIST benchmark. The result is a solid, well-generalizing foundation for banking, postal, education, and document-processing applications, with a clear roadmap toward CNN-based accuracy gains and production deployment.

---

Vaidehi Aggarwal • VIPS, GGSIPU • July 2026

# Thank You

---

Vaidehi Aggarwal